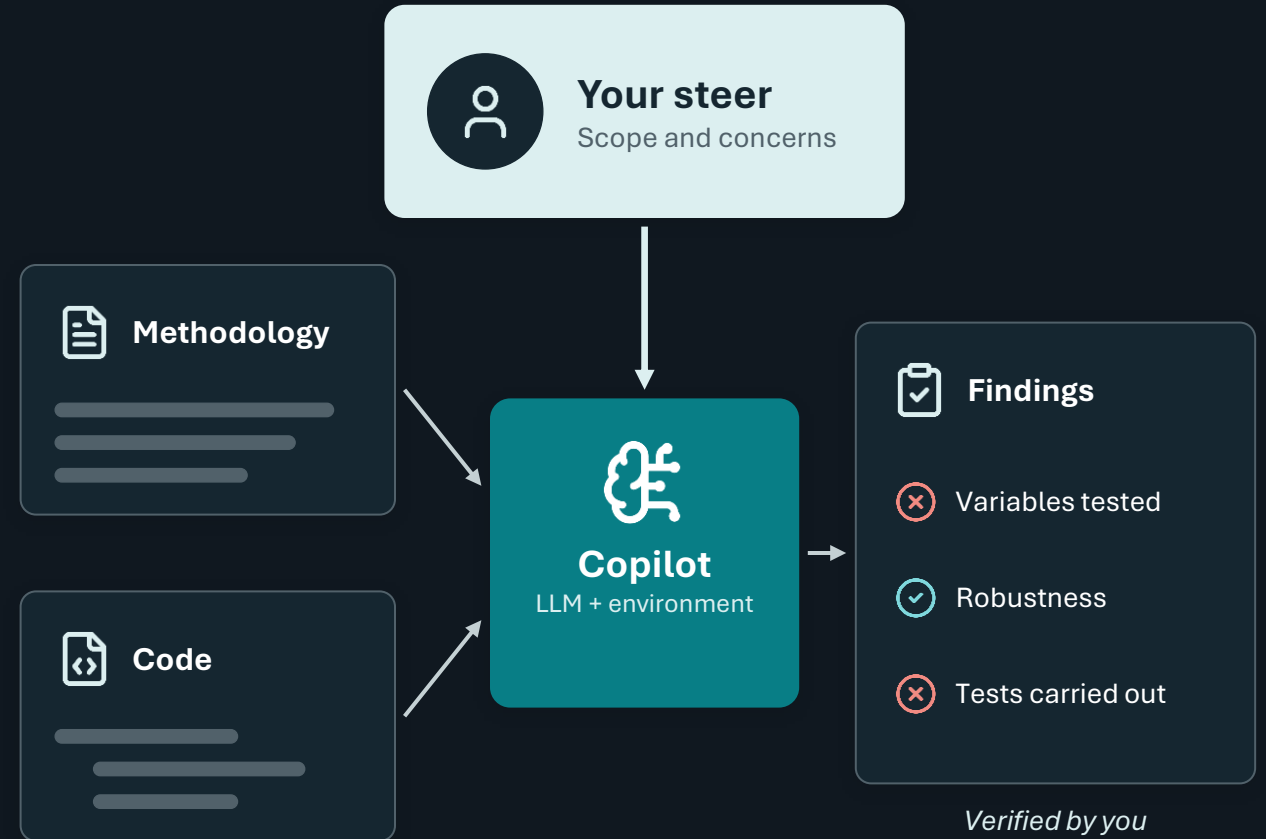


How to use GitHub Copilot

A practical workflow for
technical model review



Three things to know before you start

How long the advice holds, who it is for, and what never goes into an LLM.



A moving target

These recommendations can be outdated with the next model release.



Target audience

First-time adopters of GitHub Copilot.



PRIVACY AND CONFIDENTIALITY

Never feed confidential or private information into an LLM unless you have approval.

When in doubt, ask for permission. It is better to be safe than sorry.

The same LLM can perform at very different levels

In Copilot you use an assistant: an LLM plus settings, inputs, capabilities, and hidden instructions.

MINIMAL ENVIRONMENT

- ☐ **Same LLM**
- ☐ Low
- ☐ Pasted excerpt, files named in prose
- ☐ Scanned PDF, tables and formulas lost
- ☐ None, cannot run anything
- ☐ Off, relies on training data
- ☐ Basic chat defaults

Mostly disabled



WHAT THE ASSISTANT IS MADE OF



LLM

Core language and reasoning



Thinking effort

How much it reasons



Context

Files, history, instructions



OCR / document parsing

What document content gets through



Tool access

What it can inspect or run



Internet access

What it can look up



Hidden system prompt

Product rules you cannot see

PERFORMANCE LEVEL

WELL-EQUIPPED ENVIRONMENT

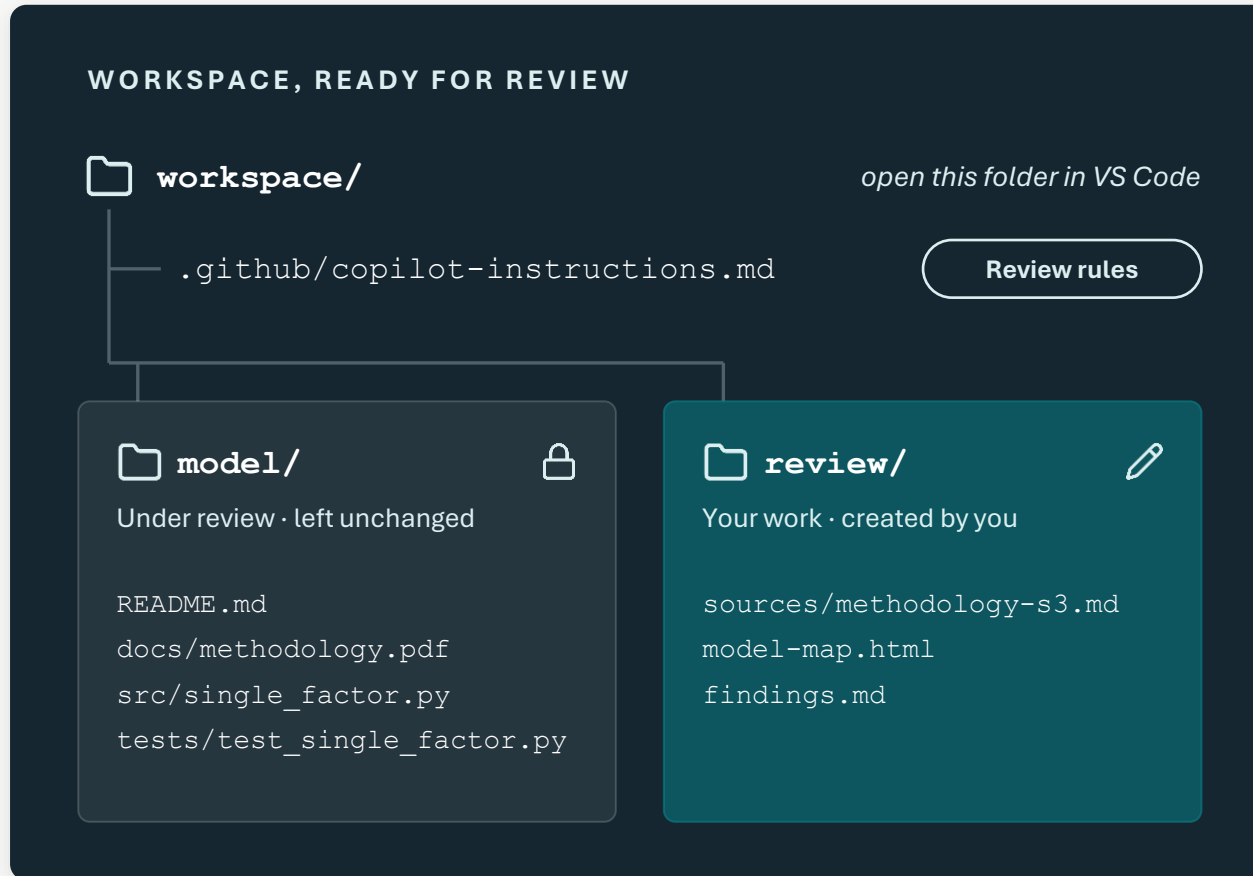
- ✓ **Same LLM**
- ✓ High where the question needs it
- ✓ Attached README, spec, code, and tests
- ✓ Markdown with tables and formulas intact
- ✓ Searches the repo and runs tests
- ✓ Retrieves current documentation
- ✓ Agent-mode defaults

Full power



Make the evidence explicit before you ask anything

Copilot can only review what it receives. Set up the workspace and the documents before the first question.



Keep model and review apart

Open the parent folder in VS Code. The model stays unchanged, and your sources, map, and findings live in review/.



Convert documents to Markdown

Keep headings, tables, formulas, and page references. Check thresholds and table values against the original PDF.

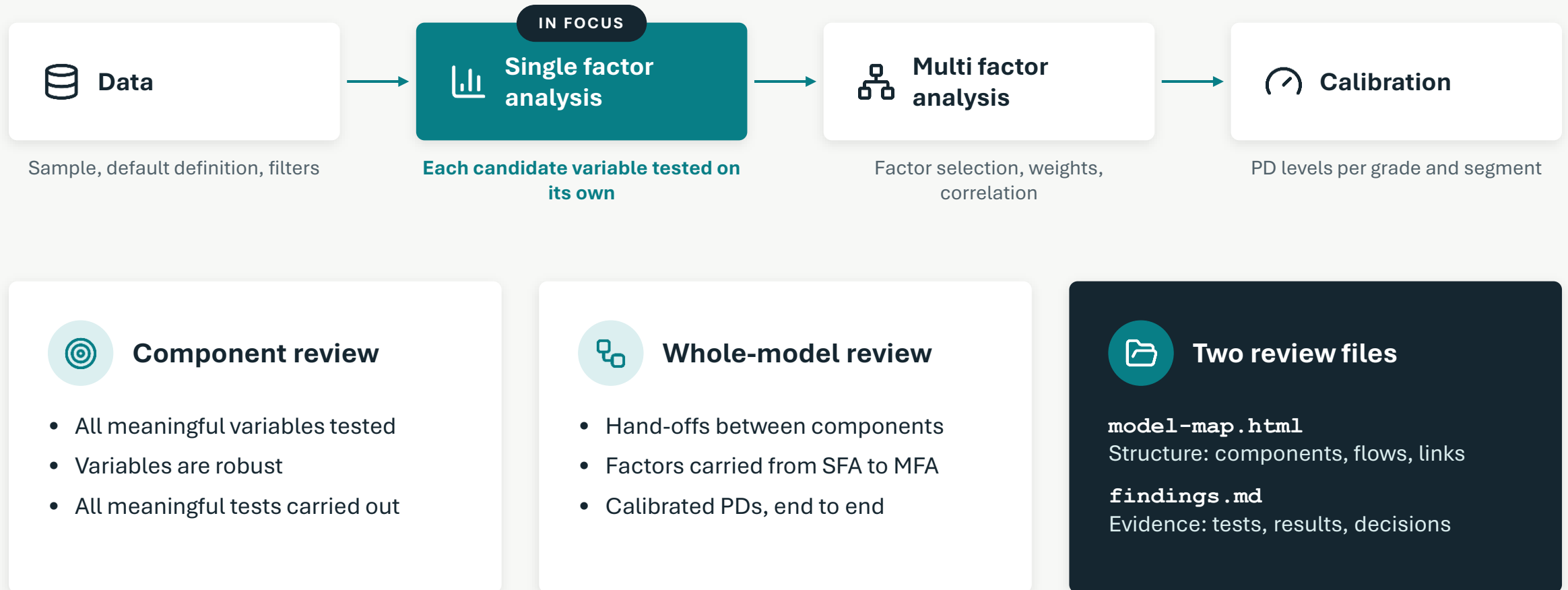


Separate facts from review rules

The model's README holds project facts. Your review rules go in one instructions file at the workspace root.

Map the model, then review one component at a time

A flowchart exposes the dependencies that a long document can hide.



Give Copilot one component and your high-level concerns

Attach the code and its specification section together, then let Copilot propose what might be wrong.



Copilot Chat · scoped review prompt

CONTEXT

#review/sources/methodology-s3.md

#model/README.md

#model/src/single_factor.py

#model/tests/test_single_factor.py

REQUEST

“Review the single factor analysis against methodology section 3. Check that all meaningful variables are tested and robust, and that all meaningful tests are carried out.”

EXPECTED OUTPUT

“Propose hypotheses, check each against code and document, and report evidence, a minimal test, and impact.”

Route by difficulty

Presenter’s preference. Availability depends on plan and setup.

Explain a component or compare docs with code

Claude Opus 5.5

Medium

Investigate a subtle bug across components

Claude Opus 5.5

High

Resolve a hard open question or get a second view

GPT-6 Astra

High



Comparing models? Keep effort, context, parsing, tools, and instructions identical.

Copilot proposes the hypotheses, and evidence decides

Illustrative case: the single factor analysis of a retail PD model, reviewed against methodology section 3.

COMPONENT IN FOCUS Single factor analysis

Methodology §3 sets the candidate variables, robustness checks, and required tests.

1 Scope

YOU

One component
single_factor.py + §3

YOUR HIGH-LEVEL STEER

“Check that all meaningful variables are tested and robust, and that all meaningful tests are carried out.”

2 Hypotheses

COPILOT

UNVERIFIED

- **H1** Have all meaningful variables been tested?
- **H2** Are the variables robust?
- **H3** Have all meaningful tests been carried out?

3 Evidence

COPILOT RUNS, YOU CHECK

- ✗ **H1** Only 26 of 30 candidate variables in §3.1 tested.
- ✓ **H2** Results hold across time windows and subsamples.
- ✗ **H3** Monotonicity test in §3.4 not run.

4 Record

YOU DECIDE

[review/findings.md](#)

F-01 4 variables untested
F-02 No monotonicity test
H2: no issue found
Next: test 4 variables, run monotonicity test

✓ **You steer, Copilot proposes.** A hypothesis becomes a finding only when it is traced, reproduced, and recorded.

Record the result, then choose the next chat deliberately

findings.md is the durable record. The chat history is only working memory.

BEFORE YOU SWITCH, UPDATE

review/findings.md

Handoff

- Goal
- Finding IDs: F-01, F-02
- Verified facts
- Open questions
- Source paths
- Next bounded task

Where should the next question go?

Follow-up on the same issue	→	Same chat	Continues the conversation
Try an alternative approach	→	Fork	Keeps history up to a chosen point
Long, related investigation	→	/compact	Summarizes older content, may lose detail
A new component or task	→	New chat	Starts with fresh history

Watch usage

Tokens measure volume. AI credits measure billing.



Context

History, files, and tool output build up



Reasoning

Higher effort uses more thinking tokens



Actions

Searches, runs, and rewrites add work

Reference links

GitHub and VS Code documentation, checked 3 October 2026



Supported Copilot models

GitHub Docs · docs.github.com



Model picker and thinking effort

VS Code Docs · code.visualstudio.com



Context and file references

VS Code Docs · code.visualstudio.com



Repository custom instructions

GitHub Docs · docs.github.com



Usage, forking, and compaction

VS Code Docs · code.visualstudio.com



Account usage monitoring

GitHub Docs · docs.github.com

 Model choices reflect the presenter's workflow. Recheck availability and pricing before training.